Compiler Design

Lecture-01

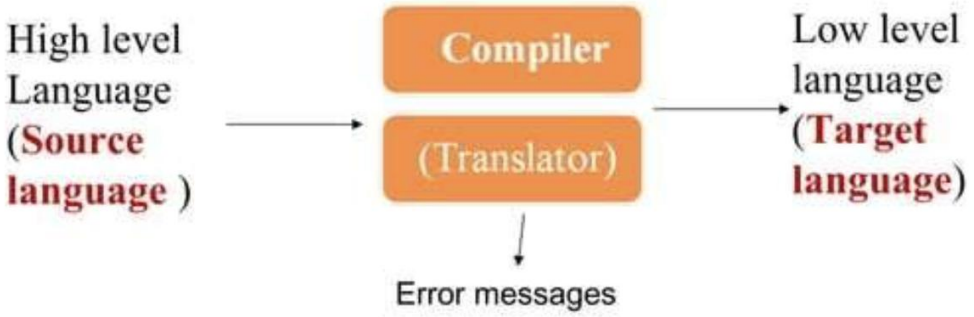
Sakurah Ismail
Course: CSE 4111

Outline

- What is Compiler?
- Why do we need a compiler?
- Different type of translator.
- Difference between compiler and interpreter.
- Compiler Architecture
- Phases of Compiler

Compiler

A compiler acts as a translator, transforming human-oriented programming languages (high level language) into computer-oriented machine languages (low level language)



Why do we need a Compiler?

- We need compiler' because computers cannot understand human language, and humans cannot understand computer language.
- Compilers are like translators that convert human-readable/writable code into assembly code, which is very difficult to understand and use. The compiler then converts the assembly code into binary code

Translator

A translator is a computer program which converts source language to target language.

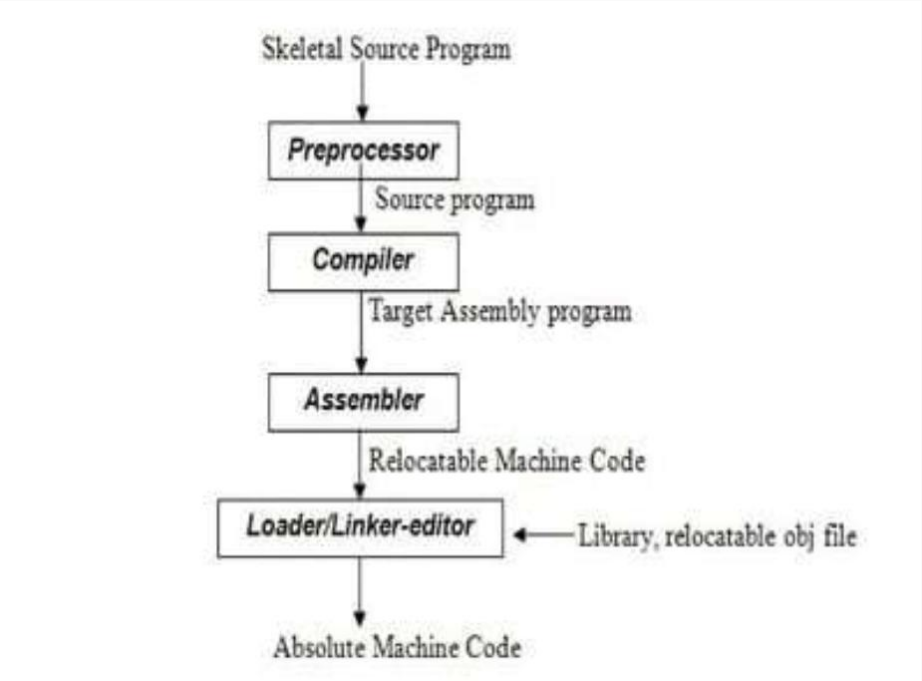
Types of Translator :

- Preprocessor
- Compiler
- Interpreter
- Assembler

Difference Between Compiler & Interpreter

COMPILER	INTERPRETER
Scans the entire program and translates it as a whole into machine code.	Translates program one statement at a time.
Fast execution	Slow execution
Memory Requirement is **more**	**Memory Requirement** is **Less**
Intermediate Object Code is **Generated**	**No** Intermediate Object Code is **Generated**
Errors are displayed after **entire program** is checked	**Errors** are displayed for **every instruction** interpreted (if any)

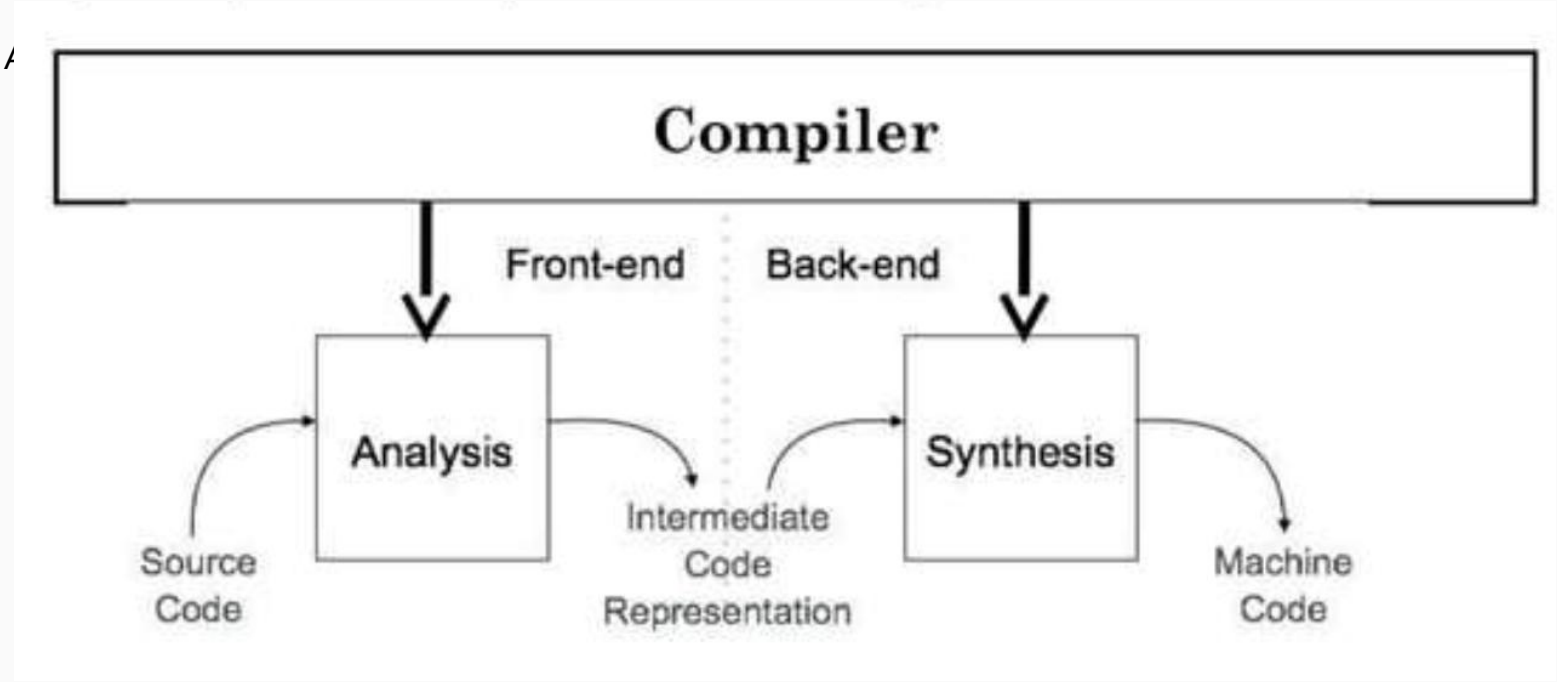
Language Processing System



Program Execution Steps

1. User writes a program in C language (high - level language.)
2. The C compiler, compiles the program and translates it to assembly program (low - level language).
3. An assembler then translates the assembly program into machine code object
4. A linker tool is used to link all the parts of the program together for execution
5. Executable machine code.
6. A loader loads all of them into memory and then the program is executed.

Compiler Architecture



Front-End → Analysis Phase

Input: **Source Code**

Output: **Intermediate Code Representation (IR)**

What happens in Analysis?

- Breaks the source code into tokens (**Lexical Analysis**)
- Checks grammar rules (**Syntax Analysis**)
- Checks meaning and type errors (**Semantic Analysis**)
- Generates an **Intermediate Representation (IR)**

Intermediate Code Representation (IR)

- A machine-independent representation of the program.
- Acts as a bridge between Front-end and Back-end.
- Makes the compiler portable.
- Allows optimization before final code generation.

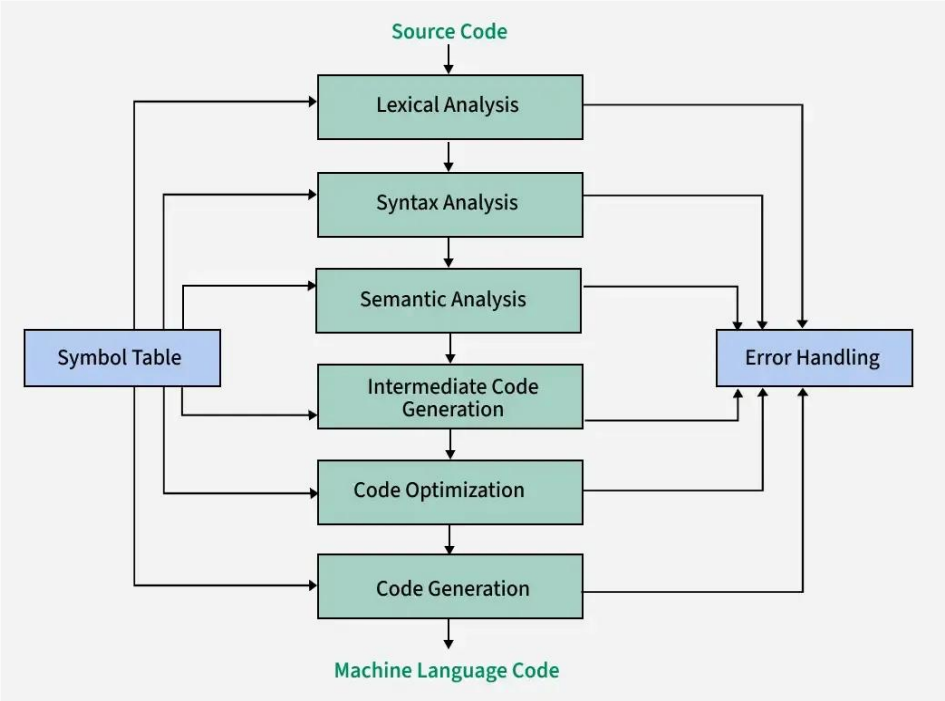
Back-End → Synthesis Phase

- Also called the **Synthesis Phase**.
- Input: **Intermediate Code**
- Output: **Machine Code**

What happens in Synthesis?

- Optimizes the intermediate code
- Allocates memory and registers
- Generates target machine instructions
- Produces final executable machine code

Phases of a Compiler



Lexical Analysis

- Responsible for converting the raw source code into a sequence of tokens.

Example: int x = 10;

The lexical analyzer would break this line into the following tokens:

- **int** - Keyword token (data type)
- **x -** Identifier token (variable name)
- **= -** Operator token (assignment operator)
- **10** - Numeric literal token (integer value)
- **; -** Punctuation token (semicolon, used to terminate statements)

Syntax Analysis or Parsing

- This phase ensures that the code follows the correct grammatical rules of the programming language.

- If the source code is not structured correctly, the syntax analyzer will generate errors

To represent the structure of the source code, syntax analysis uses parse trees or syntax trees.

- **Parse Tree:** A parse tree is a tree-like structure that represents the syntactic structure of the source code. Each branch in the tree represents a production rule of the language, and the leaves represent the tokens.
- **Syntax Tree:** A syntax tree is a more abstract version of the parse tree. It represents the hierarchical structure of the source code but with less detail, focusing on the essential syntactic structure.

Semantic Analysis

Semantic analysis is the phase of the compiler that ensures the source code makes sense logically.

- It checks whether the program has any semantic errors, such as type mismatches or undeclared variables.
- This phase ensures that the source code follows the rules of the programming language in terms of its logic and data usage.

Example: int a=5, float b=3.5 , a=a+b

Intermediate Code Generation

- Intermediate code is a form of code that lies between the high-level source code and the final machine code.
- **Simplifying Optimization:** Intermediate code simplifies the optimization process by providing a clearer, more structured view of the program. This makes it easier to apply optimization techniques such as:
    - **Dead Code Elimination:** Removing parts of the code that don't affect the program's output.
    - **Loop Optimization:** Improving loops to make them run faster or consume less memory.
    - **Common Subexpression Elimination:** Reusing previously calculated values to avoid redundant calculations.

Code Optimization

- Code Optimization is the process of improving the intermediate or target code to make the program run faster, use less memory, or be more efficient, without altering its functionality.

- **Common Techniques**:
  - **Constant Folding**: Precomputing constant expressions.
  - **Dead Code Elimination**: Removing unreachable or unused code.
  - **Loop Optimization**: Improving loop performance through invariant code motion or unrolling.
  - **Strength Reduction**: Replacing expensive operations with simpler ones.
- Example:

Code Before Optimization	Code After Optimization

for (int j = 0 ; j < n ; j ++)
{
x = y + z ;
a[j] = 6 * j;
}
	x = y + z ;
	for (int j = 0 ; j < n ; j ++)
	{
	a[j] = 6 * j;
	}

Code Generation

- Code Generation is the final phase of a compiler, where the intermediate representation of the source program is translated into machine code or assembly code.

- The source code written in a higher-level language is transformed into a lower-level language that results in a lower-level object code, which should have the following minimum properties:
  - It should carry the exact meaning of the source code.
  - It should be efficient in terms of CPU usage and memory management.

Three Address Code	Assembly Code

t1 = c * d
t2 = b + t1
a = t2
	LOAD R1, c ; Load the value of 'c' into register R1
	LOAD R2, d ; Load the value of 'd' into register R2
	MUL R1, R2 ; R1 = c * d, store result in R1
	LOAD R3, b ; Load the value of 'b' into register R3
	ADD R3, R1 ; R3 = b + (c * d), store result in R3
	STORE a, R3 ; Store the final result in variable 'a'

Thank You